Controle de clientes, produtos e pedidos

Aluno: Gabriel Vieira de Abreu
RU: 5033399
Disciplina: Desenvolvimento Web Back-End
Professora: Luciane Kanashiro, Me.
Data do estudo de caso: 12 de outubro de 2026

Apresentação

Este trabalho apresenta uma API REST para uma pequena loja de pão chinês. O projeto organiza os cadastros de clientes e produtos e registra pedidos com um único produto e uma quantidade definida. A implementação usa Java, Spring Boot, Spring Data JPA e banco relacional H2, opção expressamente permitida no enunciado.

Situação deste documento

O relatório e o código foram preparados como base de entrega. **A versão final depende do RU do aluno, das capturas reais do Postman e do endereço de um repositório publicado.** Os exemplos JSON apresentados são exemplos documentados, não capturas de testes. O enunciado informa que a ausência de prints torna a entrega incompleta.

Conteúdo

1. Estudo de caso; 2. Casos de uso UML; 3. Entidades e banco; 4. Arquitetura e regras; 5. Endpoints; 6. Exemplos JSON; 7. Execução e testes; 8. Evidências e repositório; 9. Considerações e referências.

1. Estudo de caso

Baozi Store | Desenvolvimento Web Back-End

A Baozi Store é uma pequena loja de bairro especializada na venda de baozi, um pão chinês recheado e preparado no vapor. No início de suas atividades, a loja oferece apenas uma variedade: baozi de carne suína. O atendimento é simples, mas o proprietário precisa organizar os cadastros para consultar quem comprou, qual produto foi escolhido e quantas unidades foram solicitadas.

Para esse cenário, foi desenvolvido um sistema de controle acessado por uma API REST. O cadastro do cliente identifica o comprador e a data em que ele passou a ser cliente. O cadastro do produto informa o nome, o preço unitário e a disponibilidade. O pedido relaciona esses dois cadastros e guarda a quantidade comprada.

No dia 12 de outubro de 2026, o cliente **GabrielVieiraDeAbreu5033399** realiza seu cadastro. O nome foi unido ao RU 5033399, conforme solicitado no enunciado. O produto cadastrado é **Baozi de carne suína**, com preço unitário de **R$ 8,50** e disponibilidade definida como verdadeira. Em seguida, o cliente faz um pedido de **cinco unidades**.

O registro do pedido guarda o identificador do cliente, o identificador do produto e a quantidade 5. A compra corresponde a R$ 42,50, considerando cinco unidades de R$ 8,50. Esse valor serve apenas para contextualizar a situação: o total não é um campo persistido nem um endpoint adicional no projeto.

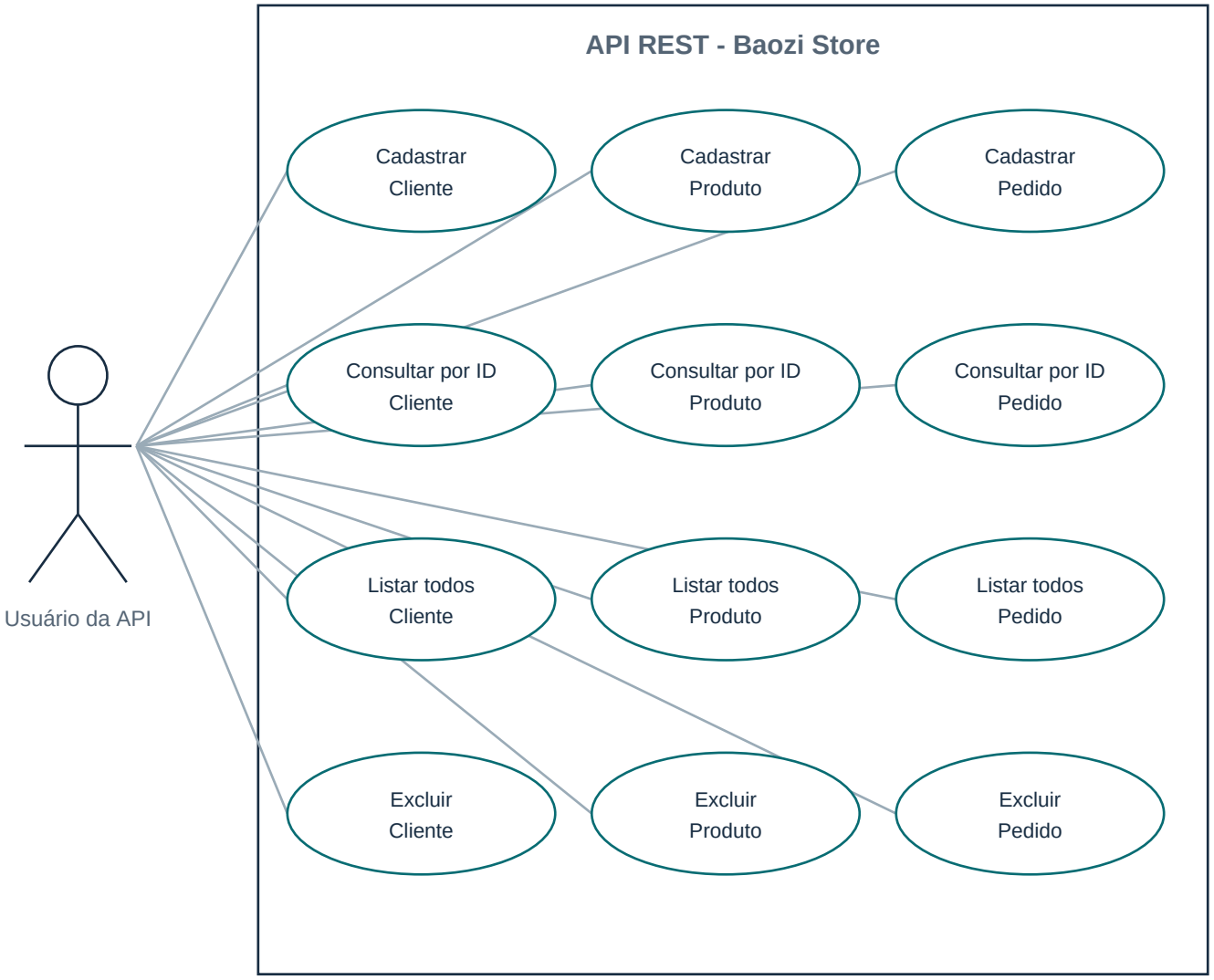
Informação	Valor usado no cenário
Cliente	GabrielVieiraDeAbreu5033399
Data de cadastro	2026-10-12
Produto	Baozi de carne suína
Preço unitário	8.50 no JSON; R$ 8,50 no texto
Disponibilidade	estoque: true
Quantidade do pedido	5

O sistema não realiza pagamentos nem controla entregas. Ele oferece as operações necessárias para cadastrar, listar, consultar por ID e excluir as três entidades. Os dados acima devem aparecer nos POSTs e nas consultas do Postman para manter a correspondência entre a descrição e as evidências.

2. Diagrama de caso de uso UML

Baozi Store | Desenvolvimento Web Back-End

O ator principal é o Usuário da API, representando a pessoa que envia requisições pelo Postman ou por outro cliente HTTP. O retângulo delimita o sistema; cada elipse representa uma funcionalidade. As associações indicam a interação do ator com cada caso de uso.



São 12 casos de uso, correspondentes aos 12 endpoints obrigatórios. Não há caso de uso de atualização porque PUT é opcional e não foi implementado. O pedido contém um único produto; por isso, não existe uma entidade de itens de pedido neste escopo.

3. Entidades e banco de dados

Baozi Store | Desenvolvimento Web Back-End

Entidade / campo	Tipo Java	Persistência e finalidade
Cliente.id	Long	Chave primária gerada automaticamente
Cliente.nome	String	Obrigatório, até 120 caracteres
Cliente.clienteDesde	LocalDate	Data obrigatória; não pode ser futura
Produto.id	Long	Chave primária gerada automaticamente
Produto.nome	String	Obrigatório, até 120 caracteres
Produto.preco	BigDecimal	DECIMAL(10,2); mínimo 0,01
Produto.estoque	Boolean	Obrigatório; disponibilidade do produto
Pedido.id	Long	Chave primária gerada automaticamente
Pedido.cliente	Cliente	ManyToOne; FK cliente_id obrigatória
Pedido.produto	Produto	ManyToOne; FK produto_id obrigatória
Pedido.quantidade	Integer	Obrigatória e maior ou igual a 1

Relacionamentos

Um cliente pode possuir zero ou vários pedidos, mas cada pedido possui exatamente um cliente. Um produto pode estar associado a zero ou vários pedidos, mas cada pedido possui exatamente um produto. A entidade Pedido usa relacionamentos JPA; na entrada e na saída JSON, os vínculos são representados pelos campos clienteId e produtoId, por meio de DTOs.

Configuração adotada

O H2 usa a URL jdbc:h2:file:./data/baozi, persistindo os registros em arquivo. O Hibernate cria ou ajusta as tabelas com ddl-auto=update. As tabelas são clientes, produtos e pedidos. As chaves estrangeiras impedem pedidos sem cadastros associados. Existe ainda um perfil MySQL opcional, configurado por variáveis de ambiente.

4. Arquitetura e regras de negócio

Baozi Store | Desenvolvimento Web Back-End

Pacote	Responsabilidade
model	Entidades Cliente, Produto e Pedido e mapeamento JPA
repository	Interfaces JpaRepository e consultas de vínculo
controller	Recebe HTTP, valida dados e produz respostas JSON
dto	PedidoRequest e PedidoResponse, com IDs no JSON
exception	Padroniza erros de validação, formato e integridade

A organização segue o padrão MVC do Spring no contexto de uma API. Os controllers recebem as requisições; o modelo representa os dados; a resposta é serializada em JSON pelo Spring, sem páginas HTML. Os repositories usam Spring Data JPA para persistência. Como o projeto é pequeno, as regras ficam nos controllers; não foi adicionada uma camada service apenas para repassar chamadas.

Regras implementadas

O cadastro rejeita nomes vazios ou muito longos. A data clienteDesde é obrigatória e não aceita uma data futura. O preço utiliza BigDecimal para representar valores decimais, deve ser positivo e aceitar no máximo duas casas decimais. O campo estoque não pode ser nulo.

Para registrar um pedido, clienteId e produtoId precisam ser positivos e corresponder a registros existentes. A quantidade deve ser um número inteiro maior ou igual a 1. Um produto com estoque=false não pode receber novos pedidos. A criação e a exclusão de pedidos são executadas em transações.

A exclusão de cliente ou produto é bloqueada quando há um pedido associado. A API retorna 409 e orienta a excluir o pedido primeiro. Isso protege o relacionamento sem apagar pedidos automaticamente. Quando o ID não existe, a consulta ou exclusão retorna 404.

Significado de estoque

O enunciado sugere estoque como Boolean. Portanto, esse campo indica disponibilidade, não quantidade física. Não há baixa automática de unidades nem verificação de saldo. O código mantém essa decisão de forma explícita para não atribuir ao sistema funcionalidades que ele não possui.

5. Especificação dos endpoints

Baozi Store | Desenvolvimento Web Back-End

Endereço base local: **http://localhost:8080**. Requisições POST usam Content-Type: application/json. O ID é gerado pelo banco e não deve ser enviado nos cadastros. As listas retornam arrays JSON ordenados por ID.

Método	Rota	Operação / sucesso
POST	/clientes	Criar cliente | 201 + Location
GET	/clientes	Listar registros | 200
GET	/clientes/{id}	Consultar um registro | 200
DELETE	/clientes/{id}	Excluir registro | 204 sem corpo
POST	/produtos	Criar produto | 201 + Location
GET	/produtos	Listar registros | 200
GET	/produtos/{id}	Consultar um registro | 200
DELETE	/produtos/{id}	Excluir registro | 204 sem corpo
POST	/pedidos	Criar pedido | 201 + Location
GET	/pedidos	Listar registros | 200
GET	/pedidos/{id}	Consultar um registro | 200
DELETE	/pedidos/{id}	Excluir registro | 204 sem corpo

Respostas de erro

Status	Situação
400 Bad Request	Campo inválido, data inválida, JSON incorreto ou ID em formato inválido
404 Not Found	Cliente, produto ou pedido não encontrado
409 Conflict	Produto indisponível ou tentativa de exclusão de cadastro vinculado

O POST retorna o recurso criado e o cabeçalho Location com a rota de consulta. Uma lista sem registros retorna [] com 200. Uma exclusão bem-sucedida retorna 204 com corpo vazio. Não há login ou autorização no escopo acadêmico local.

6. Exemplos de requisições JSON

Exemplos ilustrativos. Os IDs 1 abaixo pressupõem um banco novo. Use os IDs realmente devolvidos pelos POSTs. Na coleção, isso é feito automaticamente pelas variáveis clienteId, produtoId e pedidoId.

POST /clientes

```
{
  "nome": "GabrielVieiraDeAbreu5033399",
  "clienteDesde": "2026-10-12"
}
```

Resposta esperada: 201; corpo com id, nome e clienteDesde. Exemplo de Location: /clientes/1. O nome do cliente já contém o RU informado.

POST /produtos

```
{
  "nome": "Baozi de carne suína",
  "preco": 8.5,
  "estoque": true
}
```

Resposta esperada: 201; corpo com id, nome, preco e estoque. O valor 8.5 representa o mesmo preço que 8.50.

POST /pedidos

```
{
  "clienteId": 1,
  "produtoId": 1,
  "quantidade": 5
}
```

Resposta esperada: 201; exemplo de corpo: {"id":1,"clienteId":1,"produtoId":1,"quantidade":5}. Os DTOs mantêm a representação simples, sem serializar o cadastro inteiro dentro de cada pedido.

7. Execução e verificação

Como iniciar no computador

Instale um JDK 17 ou superior. Abra a pasta baozi-store, que contém o arquivo pom.xml. No terminal do Windows, confira java -version e javac -version. O comando javac verifica que foi instalado um JDK, e não apenas um ambiente de execução Java. O Maven Wrapper incluído no projeto baixa a versão necessária automaticamente.

```
mvnw.cmd clean test
mvnw.cmd spring-boot:run
```

A API usa a porta 8080. Se ela estiver ocupada, feche o processo que a utiliza ou ajuste server.port e a variável baseUrl do Postman. O banco H2 será salvo na pasta data do diretório em que a aplicação foi iniciada.

Testes automatizados

O projeto inclui dois testes de integração com Spring Boot e MockMvc. O primeiro percorre criação das três entidades, listagens, consultas, bloqueio de exclusão com vínculos e exclusão na ordem correta. O segundo verifica dados inválidos, referências inexistentes, produto indisponível e identificador não numérico. O banco de teste é H2 em memória, separado do banco persistente usado pela aplicação.

Resultado nesta preparação: **testes executados e aprovados; consulte o registro de verificação no pacote.** A existência de testes no código não deve ser confundida com um teste aprovado. Os testes automatizados também não substituem os prints exigidos do Postman.

Testes manuais no Postman

Importe a coleção incluída no projeto. Preencha nomeCliente com seu nome seguido do RU real. Execute primeiro os POSTs 01, 02 e 03, depois as listagens 04 a 06 e as consultas 07 a 09. Capture todas essas telas antes dos DELETEs. Por fim, execute 10, 11 e 12 para excluir pedido, cliente e produto, respectivamente.

8. Evidências e repositório

Baozi Store | Desenvolvimento Web Back-End

Seção pendente de preenchimento com evidências reais. Não foram inseridas imagens simuladas do Postman. Cada captura deve permitir conferir método, URL, status e resposta; os POSTs também devem mostrar os dados enviados. Identifique cada imagem com uma legenda e preserve a leitura do texto.

Arquivo sugerido	Conteúdo da captura	Status esperado
01.png	POST /clientes com nome + RU	201
02.png	POST /produtos: baozi, 8.50, true	201
03.png	POST /pedidos: IDs válidos, quantidade 5	201
04.png	GET /clientes	200
05.png	GET /produtos	200
06.png	GET /pedidos	200
07.png	GET /clientes/{id}	200
08.png	GET /produtos/{id}	200
09.png	GET /pedidos/{id}	200
10.png	DELETE /pedidos/{id}	204
11.png	DELETE /clientes/{id}	204
12.png	DELETE /produtos/{id}	204

Insira os 12 prints após esta seção, com as legendas correspondentes. Para exclusão, o corpo vazio faz parte do resultado correto. Se os IDs forem diferentes de 1, use os IDs obtidos na execução e mantenha a consistência entre criação e consulta.

Link do código-fonte

PENDENTE: colar aqui a URL real do repositório. O endereço precisa existir e permitir à professora acessar o projeto. Publique pom.xml, src, postman e README.md; exclua target, data, logs e senhas. O README contém os comandos de publicação e execução. Não foi criado um endereço fictício de GitHub.

9. Considerações finais e referências

Baozi Store | Desenvolvimento Web Back-End

A solução proposta atende ao controle básico da Baozi Store ao relacionar clientes, produtos e pedidos com um único produto por compra. O uso de entidades JPA e chaves estrangeiras mantém os vínculos no banco relacional. As validações ajudam a evitar cadastros incompletos, quantidades inválidas e pedidos de produtos indisponíveis.

O projeto foi mantido dentro do escopo da atividade. Não foram adicionados pagamento, entrega, autenticação ou controle numérico de estoque. Em uma evolução futura, seria possível separar regras em services, adicionar atualização de cadastros e implementar saldo de estoque, mas essas funções não são afirmadas como parte da versão atual.

Conferência antes de entregar

Item	Situação / ação necessária
Estudo de caso	Preparado com nome e RU 5033399
UML e especificação	Preparados neste relatório
Código Java e JPA	Incluídos no pacote do projeto
Banco relacional	H2 persistente padrão; MySQL opcional
Prints reais do Postman	Pendentes; inserir as 12 capturas
Repositório	Pendente; publicar e inserir URL real

Referências

KANASHIRO, Luciane. Trabalho de Desenvolvimento Web Back-End: API REST - Baozi Store. Enunciado da atividade prática fornecido na disciplina, 2026.

SPRING. Spring Boot 3.5: System Requirements. Disponível em: https://docs.spring.io/spring-boot/3.5/system-requirements.html. Consulta em 12 out. 2026. Referência utilizada para os requisitos de Java e Maven.

Documentação de apoio ao projeto: Spring Data JPA (https://docs.spring.io/spring-data/jpa/reference/); H2 Database (https://www.h2database.com); Postman Learning Center (https://learning.postman.com). Consulte essas fontes para aprofundar persistência, configuração do banco e execução de requisições.